

Python Modules (Theory + Code)

1. Introduction

A module is a Python file (.py) containing functions, variables, or classes that can be reused in other programs.

2. Why Modules?

Modules help organize code, reuse logic, and build scalable backend and AI/ML projects.

3. Creating a Module

```
# mymodule.py
def greet(name):
    return f"Hello, {name}"

def add(a, b):
    return a + b
```

4. Using a Module

```
import mymodule

print(mymodule.greet("Krishna"))
print(mymodule.add(5, 3))
```

5. Import Variations

```
# full import
import mymodule

# specific import
from mymodule import add

# alias
import mymodule as mm

# import all (not recommended)
from mymodule import *
```

6. Built-in Modules

```
import math
```

```
print(math.sqrt(16))

import random
print(random.randint(1, 10))

import datetime
print(datetime.datetime.now())
```

7. Module Search Path

```
import sys
print(sys.path)
```

8. `__name__ == '__main__'`

```
def test():
    print("Testing module")

if __name__ == "__main__":
    test()
```

9. Packages

A package is a folder containing multiple modules.

```
project/
■
■ ■ ■ mypackage/
■   ■ ■ ■ __init__.py
■   ■ ■ ■ math_utils.py
■   ■ ■ ■ string_utils.py
■
■ ■ ■ main.py
```

```
from mypackage.math_utils import add
```

10. Backend Example

```
# users.py
def create_user(name):
    return f"User {name} created"

# main.py
from users import create_user
print(create_user("Krishna"))
```

11. AI/ML Example

```
# model.py
def predict(x):
    return x * 2

# main.py
from model import predict
print(predict(10))
```

12. Best Practices

Use modules to split logic, avoid import *, and organize code into folders.

13. Practice Tasks

1. Create calculator module 2. Create string utilities 3. Use multiple modules in one project